Adapter

er · Wrap legacy XML SOAP payment gateway as PaymentProcessor · Build with V

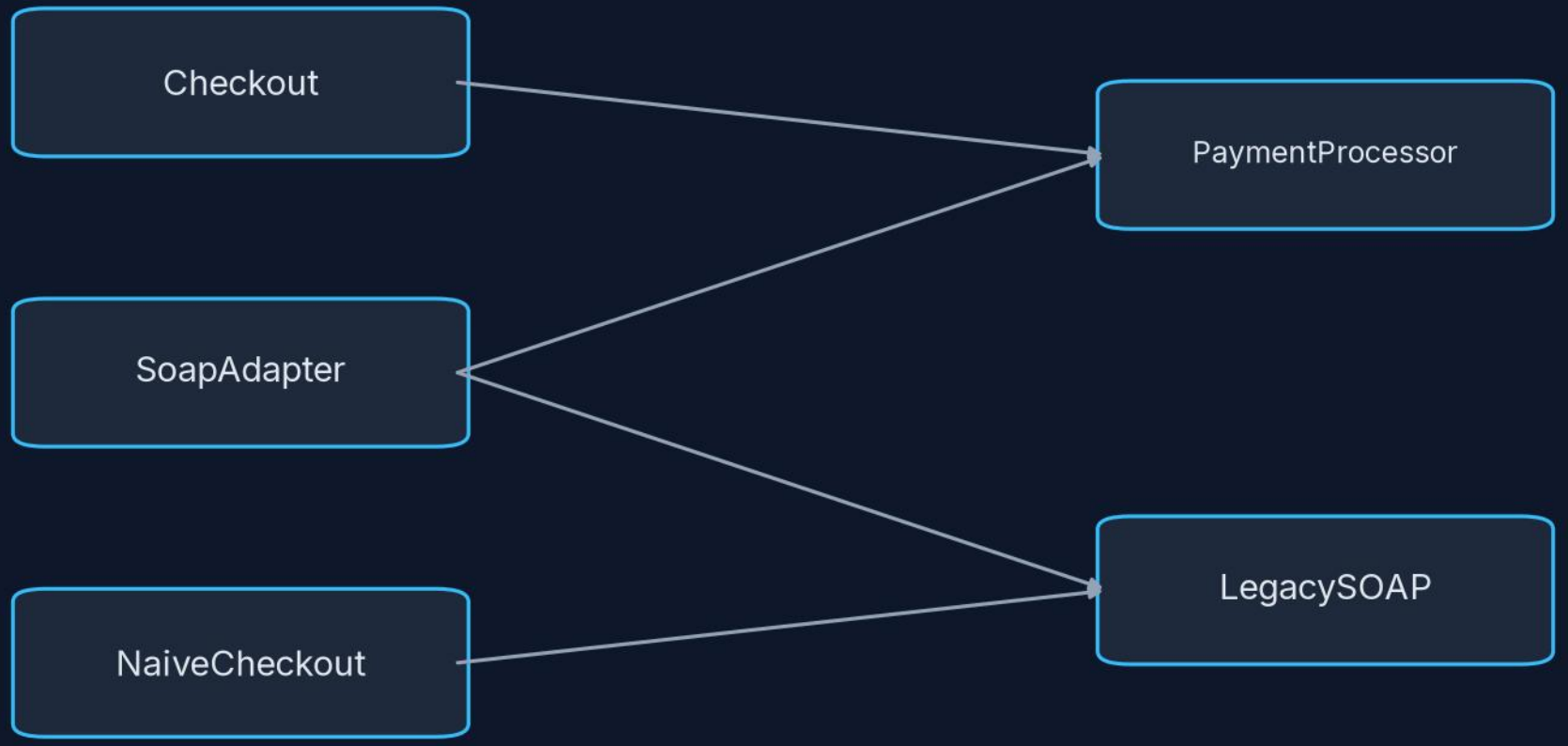
Today's Hook

- Problem: Wrap legacy XML SOAP payment gateway as PaymentProcessor
- Pattern: Adapter — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Wrap legacy XML SOAP payment gateway as PaymentProcessor
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Adapter: Wrap legacy XML SOAP payment gateway as PaymentPro

Real-World Scenario

- Running example: Wrap legacy XML SOAP payment gateway as PaymentProcessor
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.adapter.problem;

/**
 * Naive: checkout knows SOAP XML and dollar conversion.
 * Every call site duplicates envelope formatting and response scraping.
 */
public final class NaiveCheckout {
    private final LegacySoapGateway gateway = new LegacySoapGateway();

    public String pay(String customerId, int amountCents) {
        double dollars = amountCents / 100.0;
        String xml = "<Envelope><Customer>" + customerId + "</Customer><Amount>"
                + dollars + "</Amount></Envelope>";
        String ack = gateway.submitXml(xml);
        if (ack.contains("<Fault>")) {
            return "FAILED:" + ack;
        }
        int ts = ack.indexOf("<Txn>") + 5;
        int te = ack.indexOf("</Txn>");
        return "OK:" + ack.substring(ts, te);
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.adapter.problem;

/** Adapter: translates PaymentProcessor calls into LegacySoapGateway XML. */
public final class SoapPaymentAdapter implements PaymentProcessor {
    private final LegacySoapGateway gateway;

    public SoapPaymentAdapter(LegacySoapGateway gateway) {
        this.gateway = gateway;
    }

    @Override
    public PaymentResult charge(String customerId, int amountCents, String currency) {
        if (!"USD".equalsIgnoreCase(currency)) {
            return new PaymentResult(false, null, "unsupported currency " + currency);
        }
        double dollars = amountCents / 100.0;
        String xml = "<Envelope><Customer>" + customerId
                + "</Customer><Amount>" + dollars + "</Amount><Currency>USD</Currency></Envelope>";
        String ack = gateway.submitXml(xml);
        if (ack.contains("<Fault>")) {
            return new PaymentResult(false, null, ack);
        }
```

Solution (2)

```
        int ts = ack.indexOf("<Txn>") + 5;
        int te = ack.indexOf("</Txn>");
        return new PaymentResult(true, ack.substring(ts, te), "charged");
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.structural.adapter.problem;

/** Problem: checkout must talk PaymentProcessor; legacy SOAP gateway speaks XML. */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Adapter problem: legacy SOAP payment gateway ===");
        System.out.println("--- naive (controller knows SOAP XML) ---");
        System.out.println(new NaiveCheckout().pay("alice", 1999));
        System.out.println("--- adapter (PaymentProcessor) ---");
        var checkout = new CheckoutService(new SoapPaymentAdapter(new LegacySoapGateway()));
        System.out.println(checkout.checkout("alice", 1999));
        System.out.println(checkout.checkout("alice", 0));
        System.out.println(new SoapPaymentAdapter(new LegacySoapGateway()).charge("a", 100, "INR"));
    }
}
```

Walkthrough Output

```
=== Adapter problem: legacy SOAP payment gateway ===
--- naive (controller knows SOAP XML) ---
OK:SOAP-alice-1998
--- adapter (PaymentProcessor) ---
OK:SOAP-alice-1998
FAILED:<Fault>non-positive</Fault>
PaymentResult[ok=false, transactionId=null, message=unsupported currency INR]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Adapter.

Related Patterns

- Contrast Adapter with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Adapter → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.